CSC4043

Parallel Processing Lab Session III

Content

- Introduction to CUDA programming
- OpenMP vs CUDA
- CUDA installation and Verification
- First CUDA program
- Thread Identification in CUDA
- Block identification
- Performing addition

CUDA Programming Model

- CUDA - Compute Unified Device Architecture
  - NVIDIA parallel computing platform and API.
  - Extends C/C++ with GPU specific syntax, allowing developers to write code that runs across thousands of threads simultaneously.

- Host vs Device
  - Host - The CPU and its RAM
  - Device - The GPU and its dedicated memory.
- Code annotated with __global__ or __device__ runs on the GPU
- Ordinary C code runs on the CPU. Data must be explicitly transferred between Host and Device memory using cudaMemcpy.

OpenMP vs CUDA

OpenMP	CUDA
#pragma omp parallel	kernel<<<blocks, threads>>>(...)
omp_get_thread_num()	threadIdx.x + blockIdx.x * blockDim.x
omp_get_num_threads()	blockDim.x * gridDim.x
#pragma omp barrier	__syncthreads()
Shared memory (RAM)	Global memory (VRAM)
L1/L2 cache (auto)	Shared memory (manual)

Thread Hierarchy

- CUDA organises threads in a three level hierarchy.

Level	Description	Built-in Variable
Thread	The smallest unit of execution. Each thread runs the same kernel code with a unique ID.	threadIdx.x / .y / .z
Block	A group of threads that share fast Shared Memory and can synchronise with __syncthreads().	blockIdx.x / .y / .z
Grid	A collection of blocks launched together for a single kernel call.	gridDim.x / .y / .z

Memory Hierarchy

- Choosing the correct memory type is impactful performance decisions in CUDA programming.

Memory Type	Scope	Speed	Best Used For
Registers	Per-thread	Fastest	Local scalar variables
Shared Memory	Per-block (on-chip)	Very fast (~100x global)	Tile caching, inter-thread communication
Global Memory	All threads (DRAM)	Slowest (high latency)	Input/output data arrays
Constant Memory	All threads (cached)	Fast for uniform access	Read-only lookup tables

Requirements

- NVIDIA GPU (any CUDA capable, even laptop GPU works)
- Ubuntu/ Linux (recommended) or Windows with WSL2
- CUDA toolkit installed - https://developer.nvidia.com/cuda-downloads
- Basic C/ C++ knowledge

Installation and Verification

```
# Install CUDA Toolkit (Ubuntu)
sudo apt install nvidia-cuda-toolkit

# Verify installation
nvcc --version
nvidia-smi

# Check your GPU's compute capability
nvidia-smi --query-gpu=compute_cap --format=csv
```

Note: Compute capability 3.0+ is fine. 7.0+ supports Tensor Cores. Most modern laptops have 7.5+.

GPU Architecture

- How GPU differs from a CPU?
  - CPU has a few powerful cores (4-32) optimized for sequential work
  - GPU has thousands of small cores grouped into Streaming Multiprocessors (SMS), ALL designed to do the same operation on different data in parallel.
  - This is SIMT model (Single Instruction, Multiple Threads)
- Key hardware
  - SM - Streaming Multiprocessor: Fundamental compute unit. A GPU has 10-100+ SMs.
  - CUDA Core: A simple ALU inside an SM. Each SM has 32-128 cores.
  - Warp : 32 threads that execute the same instruction simultaneously.
  - Block: A group of threads you define. Always runs on one SM
  - Grid: The collection of all blocks you launch for a kernel.

First Kernel

- The CUDA Execution Model
  - A kernel is a function that runs on the GPU.
  - You launch it from CPU(host) code.
  - Every thread runs the same kernel code but works on different data based on its thread/ block index.

Hello World program

```
#include <stdio.h>

__global__ void mykernel(void){
   printf("Hello world\n");
}

int main(){
   mykernel<<<1, 10>>>();
   cudaDeviceSynchronize();
   return 0;
}
```

Code Walkthrough =>

- __global__ : Tells nvcc this runs on GPU.
- blockIdx.x, threadIdx.x, blockDim.x : Built in variables available inside every kernel. No function call is needed.
- <<<blocks, threads>>> : The launch configuration. This is CUDA specific syntax
- cudaMalloc / cudaFree : Like malloc/free but allocate GPU VRAM
- cudaMemcpy : Explicitly data transfer between CPU and GPU memory.

Note:
Unlike OpenMP where threads share RAM automatically, CUDA requires you to explicitly manage two separate memory spaces. CPU host, GPU device.

Compile and Run

```
# nvcc is the NVIDIA CUDA compiler
nvcc hello.cu -o hello.o

# Run it
./hello.o
# Expected output: c[0] = 3.0 (expected 3.0)

# Compile with optimization (always do this for benchmarks)
nvcc -O2 -o vector_add vector_add.cu

# Compile targeting a specific GPU architecture
nvcc -arch=sm_75 -O2 -o vector_add vector_add.cu
```

Hello CUDA

__global__ makes this a GPU kernel

<<<1, 10>>> Launch 1 block of 10 threads.

cudaDeviceSynchronize() Wait for all GPU threads to finish.

Thread Identification

- Each thread has a unique threadIds, x value (0-based). This is fundamental mechanism for assigning different work to different threads.

```
#include <stdio.h>

__global__ void mykernel(void) {
    int tid = threadIdx.x;   // Unique thread ID within the block
    printf("Hello world from thread %d!\n", tid);
}

int main() {
    mykernel<<<1, 5>>>();    // 1 block, 5 threads (IDs 0-4)
    cudaDeviceSynchronize();
    return 0;
}
```

Printing Block with ID

```
#include <stdio.h>

__global__ void printThreads(){
	printf("Block %d, Thread %d\n", blockIdx.x, threadIdx.x);
}

int main(){
	printThreads<<<2, 4>>>();
	cudaDeviceSynchronize();

	return 0;
}
```

Global Thread Index

```
#include <stdio.h>

__global__ void globalIndex(){
        int idx = blockIdx.x * blockDim.x + threadIdx.x;
        printf("Global Thread ID = %d\n", idx);
}


int main(){
        globalIndex<<<2, 4>>>();
        cudaDeviceSynchronize();

        return 0;
}
```

```
dulanis@wsl2328:~/Documents/Parallel_$ ./glo.o
Global Thread ID = 4
Global Thread ID = 5
Global Thread ID = 6
Global Thread ID = 7
Global Thread ID = 0
Global Thread ID = 1
Global Thread ID = 2
Global Thread ID = 3
dulanis@wsl2328:~/Documents/Parallel_$ ./glo.o
Global Thread ID = 4
Global Thread ID = 5
Global Thread ID = 6
Global Thread ID = 7
Global Thread ID = 0
Global Thread ID = 1
Global Thread ID = 2
Global Thread ID = 3
```

Addition

Goal: Add

A = [1, 2, 3, 4, 5]

B=[10, 20, 30, 40, 50]

To produce result C

Addition

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void add(int *a, int *b, int *c) {

        int idx = threadIdx.x;

        c[idx] = a[idx] + b[idx];
}
```

Addition

```
int main() {

    int h_a[5] = {1,2,3,4,5};
    int h_b[5] = {10,20,30,40,50};
    int h_c[5];

    int *d_a, *d_b, *d_c;

    cudaMalloc(&d_a, 5*sizeof(int));
    cudaMalloc(&d_b, 5*sizeof(int));
    cudaMalloc(&d_c, 5*sizeof(int));
```

Addition

```
cudaMemcpy(d_a, h_a, 5*sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(d_b, h_b, 5*sizeof(int), cudaMemcpyHostToDevice);

        add<<<1,5>>>(d_a, d_b, d_c);

        cudaMemcpy(h_c, d_c, 5*sizeof(int), cudaMemcpyDeviceToHost);

        for(int i=0; i<5; i++)
        printf("%d ", h_c[i]);

        printf("\n");

        cudaFree(d_a);
        cudaFree(d_b);
        cudaFree(d_c);

        return 0;
}
```

Vector Addition

- Goals
  - Allocate and free GPU memory using cudaMalloc and cudaFree
  - Transfer data between CPU and GPU using cudaMemcpy
  - Implement a data parallel kernel for element wise vector addition

CUDA Memory Workflow

- Every GPU computation follows the same four step memory pattern
  - Allocate memory on the GPU with cudaMalloc
  - Copy input data from CPU to GPU with cudaMemcpy (HostToDevice)
  - Launch the kernel to process data on the GPU
  - Copy results back from GPU to CPU with cudaMemcpy(DeviceToHost)

- Naming Convention
  - GPU(device) pointer variables are prefixed with d_(eg: d_a, d_b, d_c) to distinguish them from CPU(host) variables.
  - It makes code easy to read and debug.

Parallel Reduction (Sum)

- Reduction is the process of collapsing an array into a single value (e.g., sum, max, min). Naively done in O(N) serial steps, parallel reduction on a GPU with N threads takes only O(log N) steps using a tree-based approach.

Example: Summing 8 elements in 3 steps
Step 1 (stride=1): Thread 0 adds elements [0]+[1], Thread 2 adds [2]+[3], Thread 4 adds [4]+[5], Thread 6 adds [6]+[7]
Step 2 (stride=2): Thread 0 adds results [0]+[2], Thread 4 adds [4]+[6]
Step 3 (stride=4): Thread 0 adds [0]+[4] => final sum With 1024 elements, this takes only 10 steps instead of 1023!

Shared Memory & Synchronization

- Shared Memory is a small, very fast, on chip scratchpad shared by all threads in a block.
- Declared with __shared__ keyword.
- Threads must call __syncthreads() to create a barrier.
- No thread proceed past the barrier until ALL threads in the block have reached it.

Warning: Race Conditions
Without __syncthreads(), a thread may read shared_data[tid + stride] before another thread has written it. This causes a race condition producing wrong, non-deterministic results. Always synchronise after writing to shared memory.

Essential CUDA APIs

Function	Purpose	Example
cudaMalloc(ptr, size)	Allocate memory on GPU	cudaMalloc((void**)&d_a, 1024*sizeof(float))
cudaFree(ptr)	Free GPU memory	cudaFree(d_a)
cudaMemcpy(dst,src,size,dir)	Transfer data between CPU/GPU	cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice)
cudaDeviceSynchronize()	Wait for all GPU work to complete	cudaDeviceSynchronize()
__global__ void kernel()	Declare a GPU kernel callable from CPU	__global__ void myKernel(int *data)
__device__ func()	Function that runs on GPU, called from GPU	__device__ float helper(float x)
__shared__ type var	Declare shared memory within a block	__shared__ int sdata[256]
__syncthreads()	Synchronise all threads within a block	__syncthreads()
threadIdx.x	Thread index within its block (0-based)	int id = threadIdx.x
blockIdx.x	Block index within the grid (0-based)	int bid = blockIdx.x
blockDim.x	Number of threads per block	int stride = blockDim.x
gridDim.x	Number of blocks in the grid	int total_blocks = gridDim.x

End of Session III